

Provider-Auflösung

Die Provider-Schicht und die Stellen, an denen sich der Anbieter ändern lässt

Zweck

`rag_project.llm.providers` ist die Naht zwischen drei Informationsquellen: den Einstellungen aus `core/config.py`, dem Modellkatalog aus `core/models.py` und dem Anwendercode, der ein Modell wünscht. Das Ergebnis ist ein Objekt vom Typ `ResolvedTarget` — die einzige Stelle, an der diese drei Quellen zusammenlaufen. Alles nach der Auflösung arbeitet nur noch mit diesem Objekt.

Der Pfad hat sich geändert: `providers.py` liegt heute unter `llm/providers.py`. Verweise auf `rag_project.providers` sind veraltet, ebenso Verweise auf `rag_project.config` — die Konfiguration liegt unter `core/config.py`.

Was ein Ziel enthält

Feld	Bedeutung
<code>provider</code>	der aufgelöste Anbieter
<code>model</code>	der Katalogeintrag mit Kennung, Typ und Label
<code>base_url</code>	Adresse des Anbieters
<code>temperature</code>	Temperatur oder None, wenn das Modell sie nicht erlaubt
<code>max_tokens</code>	Obergrenze der Antwortlänge

Die Kurzfassung von `ResolvedTarget` nennt Kennung, Adresse, Temperatur und `max_tokens` in einer Zeile. Sie ist die Ausgabe von `--check` und von `/model` im Chatbot.

Die beiden Anbieter

Zwei Anbieter werden unterstützt. Beide sprechen dieselbe Schnittstelle, unterscheiden sich aber in Adresse und Schlüssel.

Anbieter	Umgebungsvariable	Adresse
openrouter	<code>OPENROUTER_API_KEY</code>	<code>https://openrouter.ai/api/v1</code>
openai	<code>OPENAI_API_KEY</code>	<code>https://api.openai.com/v1</code>

Die Adressen sind bewusst Konstanten im Code und keine Einstellungen: Der Entwickler kann sie ändern, der Anwender nicht. Die Zuordnung von Anbieter zu Umgebungsvariable und zu Einstellungsfeld liegt am Provider-Typ selbst — so muss keine andere Stelle diese Paare kennen.

Für OpenRouter setzt der Client zusätzlich zwei Herkunftskopfzeilen. Sie sind optional und dienen nur der Zuordnung beim Anbieter.

Wo sich der Provider ändern lässt

Der Provider lässt sich an vier Stellen bestimmen. Wie beim Modell haben sie eine feste Rangfolge, und alle vier münden in dieselbe Funktion: `resolve_target()` in `llm/providers.py`. Ein wichtiger Unterschied zum Modell: Es gibt keinen Schalter `--provider`. Der Provider folgt entweder der Einstellung `CHAT_PROVIDER` oder dem Präfix der Modellkennung.

Die Rangfolge

Ebene	Wer ändert	Beständigkeit
Präfix in der Modellkennung (Kommandozeile)	Anwender	nur für diesen einen Lauf
Argument im Python-Code	Anwender bzw. eigener Code	nur für diesen Aufruf
Umgebungsvariable des Prozesses	Betreiber	für den laufenden Prozess
.env im Projektordner	Anwender	dauerhaft, bis die Datei geändert wird

Zwei Wege, ein Ergebnis

- Der Provider kommt entweder aus `CHAT_PROVIDER` oder aus dem Präfix der Modellkennung. Ein Präfix hat dabei Vorrang vor der Einstellung: Wer `openai:gpt-4o-mini` schreibt, wählt damit den Anbieter `openai`, auch wenn `CHAT_PROVIDER` auf `openrouter` steht.

1. Kommandozeile — über das Präfix der Modellkennung

Der Anwender wechselt den Anbieter, indem er dem Modell ein Präfix gibt. Es gibt keinen eigenen Schalter dafür: Das Präfix ist die Angabe.

Befehl	Wirkung
--------	---------

Befehl	Wirkung
<code>uv run rag-project --check --model openai:gpt-4o-mini</code>	Anbieter openai für diesen Lauf erzwingen
<code>uv run rag-project --check --model openrouter:z-ai/glm-5.3</code>	Anbieter openrouter für diesen Lauf erzwingen
<code>uv run rag-project --model openai:gpt-4o-mini</code>	Statusbericht mit gewechseltem Anbieter
<code>uv run rag --ask "FRAGE" --model openai:gpt-4o-mini</code>	eine Frage über den anderen Anbieter stellen
<code>uv run chatbot --model openai:gpt-4o-mini</code>	Chatbot mit dem anderen Anbieter starten
<code>uv run rag-project --check --model z-ai/glm-5.3</code>	ohne Präfix — es entscheidet CHAT_PROVIDER

Der Wechsel verlangt einen zweiten Schlüssel

- Ein Anbieterwechsel zur Laufzeit braucht den Schlüssel des neuen Anbieters. Der Aufruf mit `openai:gpt-4o-mini` scheitert mit dem Hinweis auf `OPENAI_API_KEY`, wenn nur der OpenRouter-Schlüssel gesetzt ist.

Der Katalog einer anderen Adresse lässt sich vorher ansehen. Der Filter zeigt, welche Modelle zu welchem Anbieter gehören — nützlich, wenn man unsicher ist, ob ein Modell bei beiden Anbietern existiert.

Befehl	Wirkung
<code>uv run rag-project --models openai</code>	nur die Modelle des Anbieters openai
<code>uv run rag-project --models openrouter</code>	nur die Modelle des Anbieters openrouter
<code>uv run rag-project --models</code>	beide Anbieter untereinander

In der laufenden Chatbot-Sitzung zeigt `/model` den aktiven Provider neben dem Modell. Ändern lässt er sich dort nicht — dafür muss der Chatbot mit einem anderen Präfix neu gestartet werden.

2. Python-Code — über das Präfix oder die Settings-Instanz

Im Code gibt es zwei Wege. Der erste ist derselbe wie auf der Kommandozeile: ein Präfix an der Modellkennung. Der zweite ist eine selbst gebaute Einstellungsinstanz — damit lässt sich der Anbieter setzen, ohne eine Datei anzufassen.

Aufruf	Wirkung
<code>resolve_target("openai:gpt-4o-mini")</code>	Anbieter über das Präfix wählen
<code>build_llm("openrouter:z-ai/glm-5.3")</code>	Client des anderen Anbieters bauen
<code>Settings(chat_provider=ChatProvider.OPENAI, openai_api_key="...")</code>	Anbieter in einer eigenen Instanz setzen
<code>Settings(chat_provider=ChatProvider.OPENROUTER, openrouter_api_key="...")</code>	dasselbe für OpenRouter
<code>resolve_target("openai:gpt-4o-mini", settings)</code>	Präfix und Instanz zusammen übergeben

Der zweite Weg ist im Code der übliche, weil die Instanz den Zustand vollständig beschreibt: Wer Settings mit `chat_provider` und passendem Schlüssel baut, weiß ohne Blick in die `.env`, welcher Anbieter gilt. Die Auflösung prüft dabei, dass der Schlüssel des gewählten Anbieters vorhanden ist.

Wichtig ist die Reihenfolge bei Konflikten: Ein Präfix in der Kennung schlägt das Feld `chat_provider` der übergebenen Instanz. Wer beides angibt, bekommt den Anbieter aus dem Präfix — oder eine Meldung, wenn das Modell gar nicht zu ihm gehört.

3. Umgebungsvariable des Prozesses

`CHAT_PROVIDER` lässt sich für einen einzelnen Prozess in der Umgebung setzen. Der Wert überschreibt die `.env`, weil das Laden der `.env` keine bereits gesetzte Prozessvariable überschreibt. Mitgesetzt werden muss der Schlüssel des neuen Anbieters.

Befehl	Wirkung
<code>\$env:CHAT_PROVIDER="openai"; uv run rag-project --check</code>	PowerShell: Anbieter für diesen Aufruf
<code>CHAT_PROVIDER="openai" uv run rag-project --check</code>	Linux und macOS: vorangestellt
<code>export CHAT_PROVIDER="openai"</code>	Linux und macOS: für die Sitzung setzen
<code>\$env:CHAT_PROVIDER=""</code>	PowerShell: Wert leeren, Standard greift wieder

Der Schlüssel muss zum gesetzten Anbieter passen. Wird der Anbieter auf openai gestellt und OPENAI_API_KEY ist nicht gesetzt, bricht die Auflösung mit einer sprechenden Meldung ab, bevor der HTTP-Client überhaupt startet.

Ein unbekannter Wert wird abgewiesen: CHAT_PROVIDER nimmt nur openai oder openrouter an. Ein Tippfehler führt deshalb nicht zu einem stillen Rückfall auf den Standard, sondern zu einer Meldung.

4. .env — dauerhaft für das Projekt

Der dauerhafte Weg: CHAT_PROVIDER in der .env in der Projektwurzel setzen, dort wo auch pyproject.toml liegt. Danach gilt der Anbieter für jeden Lauf, bis die Datei wieder geändert wird.

- CHAT_PROVIDER=openrouter — der Standardwert
- CHAT_PROVIDER=openai — auf OpenAI umstellen
- Fehlt die Zeile, gilt der Default aus config.py: openrouter

Zusammen mit CHAT_MODEL ergibt sich der vollständige Zustand. Es wird immer nur der Schlüssel des aktiven Anbieters geprüft — ein reiner OpenRouter-Nutzer braucht keinen OpenAI-Schlüssel und umgekehrt. Nach einem Wechsel ist der andere Schlüssel erforderlich.

Die Einstellungen werden einmal gelesen

- get_settings() erzeugt die Einstellungen genau einmal pro Prozess. Eine Änderung an der .env wirkt deshalb nicht in einem laufenden Programm — sie wirkt beim nächsten Start.

5. Code des Projekts — ein Anbieter wird ergänzt

Soll ein dritter Anbieter möglich werden, ist das keine Einstellung, sondern eine Codeänderung. Sie betrifft vier Stellen, die zusammenpassen müssen: der Aufzählungstyp in core/config.py, seine Zuordnungen zu Adresse und Schlüsselfeld, die Standardadresse, und die Modelle im Katalog.

- Ein neuer Wert im Aufzählungstyp ChatProvider.
- Eine Zuordnung zu base_url, key_field und env_var.
- Eine neue Adresskonstante neben den bestehenden.
- Katalogeinträge mit dem neuen Anbieter, samt einem Standardmodell.

Die Tests erwarten, dass jeder Anbieter genau einen Standard hat und dass das Standardflag gesetzt ist. Ein neuer Anbieter ohne Standardmodell fällt deshalb sofort auf, statt erst beim ersten Aufruf.

Fehler beim Anbieterwechsel

Situation	Meldung
Schlüssel des Zielanbieters fehlt	MissingCredentialsError mit dem Namen der Umgebungsvariable
Modell gehört nicht zum aktiven Anbieter	ProviderError mit beiden Anbieternamen und beiden Auswegen
Kennung ohne Präfix bei mehreren Anbietern	UnknownModelError — der Katalog rät nicht
Unbekannter Wert in CHAT_PROVIDER	Validierungsfehler beim Erzeugen der Einstellungen

Befehl	Ergebnis
uv run rag-project --check --model openai:gpt-4o-mini	bei fehlendem OpenAI-Schlüssel: Hinweis auf OPENAI_API_KEY
CHAT_PROVIDER=openai uv run rag-project --check	mit gesetztem Schlüssel: OpenAI als aktiver Anbieter
uv run rag-project --check --model gpt-5.6-luna	mehrdeutig — Fehler statt Vermutung

Die Meldung bei einem fehlenden Schlüssel nennt die Umgebungsvariable und den Weg: den Wert in die .env schreiben oder in der Umgebung setzen. Die Meldung bei einem Anbieter-Mismatch nennt beide Anbieternamen und die beiden Auswege: CHAT_PROVIDER passend setzen oder ein Modell des aktiven Anbieters wählen.

Kurz: wo ändere ich was?

Ich will ...	Ebene	Beispiel
einmalig den Anbieter wechseln	Präfix am Modell	uv run rag-project --check --model openai:gpt-4o-mini
nur nachsehen, was aktiv ist	Kommandozeile	uv run rag-project --check
im eigenen Skript steuern	Einstellungsinstanz	Settings(chat_provider=ChatProvider.OPENAI, openai_api_key="...")

Ich will ...	Ebene	Beispiel
nur für einen Prozess setzen	Umgebungsvariable	<code>\$env:CHAT_PROVIDER="openai"; uv run rag-project --check</code>
dauerhaft umstellen	<code>.env</code>	<code>CHAT_PROVIDER=openai</code>
einen weiteren Anbieter möglich machen	Code	Aufzählungstyp, Adresse, Katalog erweitern

Schlüssel

Schlüssel werden in den Einstellungen als `SecretStr` gehalten. `repr()`, `str()` und Logs zeigen nur Sternchen; der Klartext existiert auf genau einem Weg. Und es gibt genau eine Stelle im Projekt, die ihn entschlüsselt: `build_llm()`. Kein anderes Modul ruft diese Methode auf.

Die Prüfung ist zweistufig. Die Einstellungen stellen sicher, dass der Schlüssel des bei Start gewählten Anbieters existiert. Die Provider-Schicht prüft zusätzlich, dass der Schlüssel des tatsächlich aufgelösten Anbieters existiert — das deckt den Wechsel zur Laufzeit ab.

Der Client

`build_llm()` erzeugt den Chat-Client aus dem aufgelösten Ziel. Übergeben werden Modellkennung, Adresse, der entschlüsselte Schlüssel, die Obergrenze der Antwortlänge, die Zeitüberschreitung und der Streaming-Schalter; die Temperatur nur, wenn sie erlaubt ist. Es gibt nur einen Client-Typ für beide Anbieter — die Unterscheidung steckt allein in Adresse und Schlüssel.

Parameter	Herkunft
<code>model</code>	<code>ResolvedTarget.model.id</code>
<code>base_url</code>	<code>ResolvedTarget.base_url</code>
<code>api_key</code>	<code>Settings.api_key_value</code> — die einzige Entschlüsselungsstelle
<code>max_tokens</code>	<code>ResolvedTarget.max_tokens</code>
<code>timeout</code>	<code>Settings.request_timeout</code>
<code>streaming</code>	Argument des Aufrufers
<code>temperature</code>	nur gesetzt, wenn <code>ResolvedTarget.temperature</code> nicht <code>None</code> ist

Statusausgabe

`provider_report()` bündelt die Ausgabe für Kommandozeile und Fehlersuche: die verdeckten Einstellungen plus das aufgelöste Modell und dessen Kurzfassung. Schlüssel erscheinen darin nur als „set (n chars)“ — nie im Klartext. Diese Funktion speist `--show-config` im Projekt-Einstieg und die Auskunft `/model` im Chatbot.

- `uv run rag-project --show-config` — aktive Einstellungen mit Fingerabdrücken
- `uv run rag-project --check` — löst das Ziel auf und zeigt Provider, Modell, Typ und Adresse
- `uv run rag --config` — dieselbe Auskunft aus der RAG-Anwendung

Testbezug

`tests/test_config_and_models.py` prüft, dass ein `SecretStr` nicht in `repr()` oder `str()` auftaucht, dass `describe_settings()` nichts preisgibt, dass das Präfix die Einstellung schlägt, dass eine mehrdeutige Kennung ohne Präfix abgewiesen wird und dass ein fehlender Schlüssel nach einem Anbieterwechsel gemeldet wird. Alles ohne Netz.